ECOLE SUPERIEUR POLYTHEQUNIQUE DE DAKAR

DEPARTEMENT GENIE INFORMATIQUE

CLASSE : L2-GLSIC

RAPPORT DE PROJET JAVA : SYSTÈME DE GESTION INTÉGRÉ DE BIBLIOTHÈQUE ET LOGISTIQUE

REALISER PAR :

ADJA MARIAMA BAL

DIEWO NDIAYE

SEYNABOU CISSE

FATOU NDIANG

INTRODUCTION

Ce rapport présente la conception et la réalisation d'une application de gestion avancée. L'objectif était de développer un système capable non seulement de gérer le cœur de métier d'une bibliothèque (livres, membres, emprunts), mais aussi d'intégrer des modules logistiques et de surveillance (gestion d'annexes, de véhicules et de capteurs environnementaux).

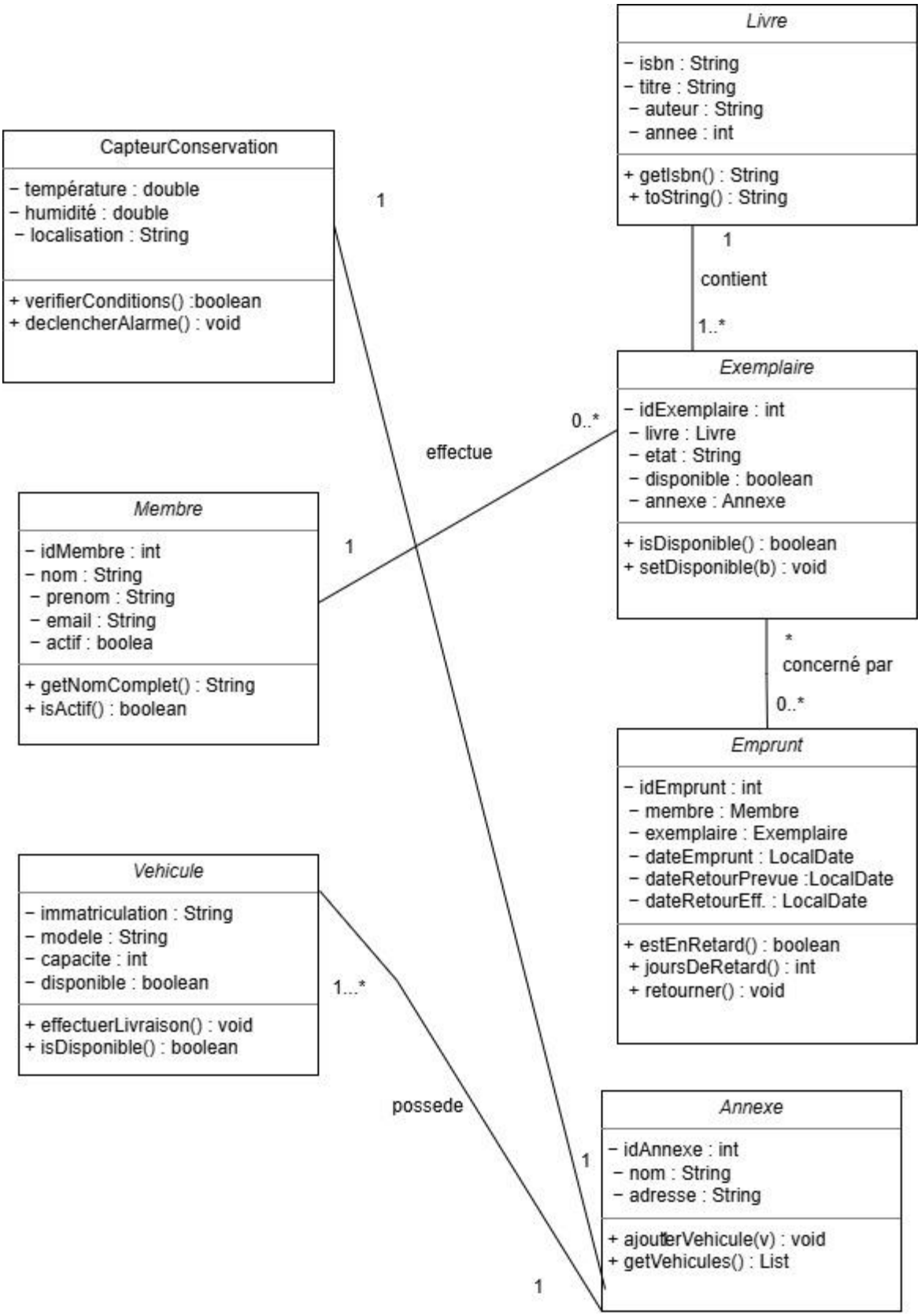
Le projet applique les principes fondamentaux de la Programmation Orientée Objet (POO) en Java, s'appuie sur une modélisation UML rigoureuse et assure la persistance des données via une base de données relationnelle MySQL.

2. PARTIE 1 : ANALYSE ET CONCEPTION (UML)

Cette phase a permis de définir la structure statique du système à l'aide d'un diagramme de classes UML.

2.1. Présentation du Diagramme de Classes

Le diagramme modélise l'ensemble des entités du système, leurs attributs, leurs méthodes et les relations qui les unissent.


Livre
− isbn : String
− titre : String
− auteur : String
− annee : int
+ getIsbn() : String
+ toString() : String
CapteurConservation
− température : double
− humidité : double
− localisation : String
+ verifierConditions() :boolean
+ declencherAlarme() : void
1
1
contient
1..*
Exemplaire
− idExemplaire : int
− livre : Livre
− etat : String
− disponible : boolean
− annexe : Annexe
+ isDisponible() : boolean
+ setDisponible(b) : void
0..*
effectue
Membre
− idMembre : int
− nom : String
− prenom : String
− email : String
− actif : boolea
+ getNomComplet() : String
+ isActif() : boolean
1
*
concerné par
0..*
Emprunt
− idEmprunt : int
− membre : Membre
− exemplaire : Exemplaire
− dateEmprunt : LocalDate
− dateRetourPrevue :LocalDate
− dateRetourEff. : LocalDate
+ estEnRetard() : boolean
+ joursDeRetard() : int
+ retourner() : void
Vehicule
− immatriculation : String
− modele : String
− capacite : int
− disponible : boolean
+ effectuerLivraison() : void
+ isDisponible() : boolean
1...*
possede
1
Annexe
− idAnnexe : int
− nom : String
− adresse : String
+ ajouterVehicule(v) : void
+ getVehicules() : List
1

2.2. Description détaillée des Entités

Nous pouvons diviser le système en trois sous-modules principaux visibles sur le diagramme :

- Module Bibliothèque :
  - Livre : Définit les informations d'un ouvrage (ISBN, titre, auteur, année).
  - Exemplaire : Représente les copies physiques d'un livre, gérant leur état, leur disponibilité, et leur rattachement à une annexe spécifique.
  - Membre : Gère les informations des adhérents (nom, prénom, email, état du compte).
  - Emprunt : Cœur de la logique métier, cette classe fait le lien entre un membre et un exemplaire, gérant les dates d'emprunt, de retour prévu et réel.

Module Logistique et Infrastructures :

- Annexe : Représente une succursale physique. Elle contient des exemplaires et possède des véhicules.
- Vehicule : Définit un véhicule (immatriculation, modèle, capacité, disponibilité).

Module de Surveillance :

- CapteurConservation : Classe technique gérant la température, l'humidité et la localisation pour assurer la sécurité des collections.

2.3. Analyse des Relations et Cardinalités

L'analyse approfondie du diagramme révèle des interactions clés :

- **Association d'Emprunt (Emprunt) :** C'est une relation centrale. Un emprunt concerne **exactement 1** membre et **exactement 1** exemplaire (cardinalités 1 à 1). Un membre peut effectuer de **0 à plusieurs (0..*)** emprunts. Un exemplaire peut être concerné par **0 à plusieurs (0..*)** emprunts au fil du temps.
- **Composition de Collection (Livre - Exemplaire) :** Un livre possède **1 à plusieurs (1..*)** exemplaires physiques.
- **Association de Possession (Annexe - Vehicule) :** Une annexe possède **0 à plusieurs (0..*)** véhicules. Un véhicule appartient à **exactement 1** annexe.
- **Association de Stockage (Annexe - Exemplaire) :** Une annexe contient **0 à plusieurs (0..*)** exemplaires. Un exemplaire est stocké dans **exactly 1** annexe.
- **Association de Surveillance (Annexe - CapteurConservation) :** Une annexe est surveillée par **0 à plusieurs (0..*)** capteurs. Un capteur surveille **exactement 1** annexe.

3. PARTIE 2 : RÉALISATION ET BACKEND (CÔTÉ JAVA)

Le développement a consisté à traduire ce modèle UML en code Java fonctionnel.

3.1. Structure du Code et Packages

Pour assurer la clarté et la maintenabilité, le code est organisé en un package nommé source. L'application suit une architecture qui sépare les données de la logique :

- **Classes Modèles :** Les classes métier (Livre, Membre, Emprunt, etc.) implémentent fidèlement les attributs et méthodes du diagramme. L'encapsulation est respectée avec des attributs privés et des getters/setters publics.
- **Logique de Contrôle :** La classe Main.java orchestrant le programme.

3.2. Interface Utilisateur et Fonctionnalités

La classe Main.java propose un menu interactif en ligne de commande. Elle permet à l'utilisateur de naviguer entre les différents modules.

› Ce PC › Bureau › JAVA_PROJET_final › source

Nom	Statut	Modifié le	Type	Taille
Annexe.class		05/04/2026 19:38	Fichier CLASS	2 Ko
Annexe		05/04/2026 13:35	Fichier source Java	1 Ko
BibliothequeDAO.class		05/04/2026 19:38	Fichier CLASS	3 Ko
BibliothequeDAO		05/04/2026 13:37	Fichier source Java	2 Ko
CapteurConservation.class		05/04/2026 19:38	Fichier CLASS	2 Ko
CapteurConservation		05/04/2026 13:38	Fichier source Java	1 Ko
ConnexionDB.class		05/04/2026 19:38	Fichier CLASS	2 Ko
ConnexionDB		05/04/2026 13:31	Fichier source Java	1 Ko
Emprunt.class		05/04/2026 19:38	Fichier CLASS	2 Ko
Emprunt		05/04/2026 13:47	Fichier source Java	1 Ko
Exemplaire.class		05/04/2026 19:38	Fichier CLASS	2 Ko
Exemplaire		05/04/2026 13:46	Fichier source Java	2 Ko
Livre.class		05/04/2026 19:38	Fichier CLASS	2 Ko
Livre		05/04/2026 13:37	Fichier source Java	1 Ko
Main.class		05/04/2026 19:38	Fichier CLASS	4 Ko
Main		05/04/2026 13:42	Fichier source Java	5 Ko
Membre.class		05/04/2026 19:38	Fichier CLASS	2 Ko
Membre		05/04/2026 13:35	Fichier source Java	1 Ko
Vehicule.class		05/04/2026 19:38	Fichier CLASS	2 Ko
Vehicule		05/04/2026 13:39	Fichier source Java	1 Ko

```
SYSTÈME DE GESTION BIBLIOTHÈQUE

MENU
1. Ajouter un membre
2. Enregistrer un emprunt
3. Vérifier conditions de conservation
4. Effectuer une livraison
0. Quitter

Votre choix : 1

--- Ajout d'un membre ---
Nom       : NDIAYE
Prénom    : DIEWO
Email     : diewo@gmail.com
```

```
Membre DIEWO NDIAYE ajouté avec succès !
```

4. PARTIE 3 : PERSISTANCE ET BASE DE DONNÉES (CÔTÉ BDD)

Une base de données relationnelle a été conçue pour stocker de manière permanente les données manipulées par l'application Java.

4.1. Schéma de la Base de Données

```sql
-- Table Emprunt (inclut la gestion de la caisse/amendes)
CREATE TABLE Emprunt (
    idEmprunt INT PRIMARY KEY AUTO_INCREMENT,
    idMembre INT,
    idExemplaire INT,
    dateEmprunt DATE,
    dateRetourPrevue DATE,
    dateRetourEff DATE NULL,
    amende_reglee DECIMAL(10,2) DEFAULT 0.0,
    FOREIGN KEY (idMembre) REFERENCES Membre(idMembre)
);
```

gestion_biblio.sql

```sql
CREATE DATABASE IF NOT EXISTS gestion_biblio;
USE gestion_biblio;

-- Table Livre (Catalogue)
CREATE TABLE Livre (
    isbn VARCHAR(20) PRIMARY KEY,
    titre VARCHAR(255) NOT NULL,
    auteur VARCHAR(255),
    annee INT
);

-- Table Membre
CREATE TABLE Membre (
    idMembre INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(100),
    prenom VARCHAR(100),
    email VARCHAR(150) UNIQUE,
    actif BOOLEAN DEFAULT TRUE
);
```

4.2. Communication Java-BDD : Le Pattern DAO

L'application utilise le motif de conception DAO (Data Access Object). La classe BibliothequeDAO agit comme une couche d'abstraction, séparant le code SQL du code Java métier. Elle implémente des requêtes SQL préparées (PreparedStatement) pour garantir la sécurité et la performance des opérations CRUD (Créer, Lire, Mettre à jour, Supprimer).

5. CONCLUSION

Ce projet a permis d'acquérir une expérience concrète et complète du développement logiciel. En partant d'une analyse UML complexe, nous avons réalisé une application Java structurée, reliée à une base de données performante. La mise en œuvre de modules aussi variés que la gestion d'emprunts et la télémétrie de capteurs a renforcé notre maîtrise des concepts de la POO et des architectures d'accès aux données. L'utilisation d'outils professionnels comme Git et GitHub a de plus permis de sécuriser et de valoriser le travail accompli.